Week 7: object-oriented programming, classes and the four pillars

MIS41110 Programming for Analytics · 2026/27 · Week 7 lecture notes

These notes cover week 7's lecture, object-oriented programming, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

Where we started

Slides, pages 1, 2, 3, 5, 6 and 7.

Poll. One word: what does `self` refer to inside a method? The object itself, the one instance the method is working on, and section 2 shows how Python hands it over.

Six weeks in, the café is a working program. `cafe_w6b.py` keeps a `MENU` dictionary, a `stock` dictionary, the revenue, the regulars and the day's orders, and eighteen loose functions pass them around: `take_order`, `complete_sale`, `review_scores`, `sales_report` and the rest. It runs, all eight menu options of it, and it fights its own shape. Week 6 pulled out one clean, authoritative function, `complete_sale`, which checks every item and only then takes the stock and the money, and it could not be wired in without rewriting half the file.

Week 7 fixes the shape. The café's things get names in the code, a menu item, an order, the register, and `complete_sale` becomes a method, `Register.sell`. The project makes the same move: its own refactor into classes is week 8's milestone, and the pattern this week builds is the one at the heart of the brief.

Recap. `groupby`, from week 6's pandas lecture: one pile per value of a key column, then one number per pile. Page 6 runs it by hand on four made-up rows of a library's loans:

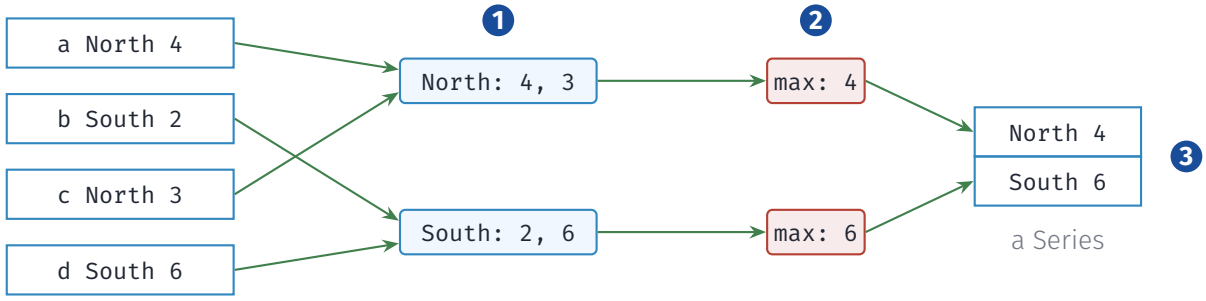
row	branch	books
a	North	4
b	South	2
c	North	3
d	South	6

With those rows in a DataFrame called `loans`, what does this call give? Walk it in its own order.

```
loans.groupby("branch")["books"].max()
```

❶ `groupby("branch")` splits the rows into one pile per branch: rows a and c go to North, rows b and d to South. ❷ `["books"]` picks the column each pile reduces, and `.max()` keeps its largest value: 4 for North, 6 for South. ❸ The results come back as a Series, one row per pile, labeled by the branch in sorted order:

```
branch
North    4
South    6
Name: books, dtype: int64
```

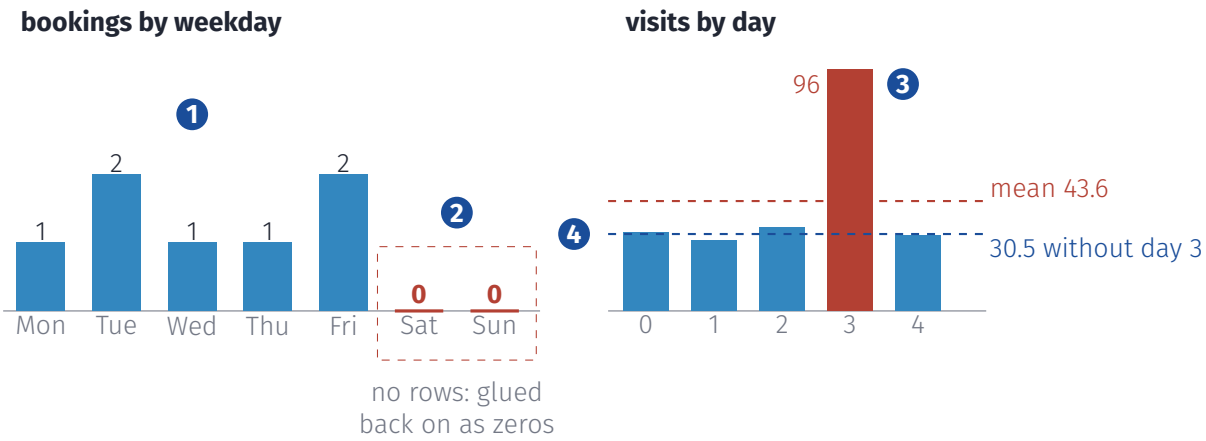


The split never changes; only the reduction does. With `.sum()` the same piles give 7 and 8, as page 6 shows, and `loans["branch"].value_counts()` counts their rows instead, 2 and 2, adding nothing up: counting rows or adding a column, the distinction page 6 names for the exam.

Page 7 takes two more of week 6's ideas to the same library, and one short session holds both:

```
>>> import pandas as pd
>>> bookings = pd.Series([0, 1, 1, 2, 3, 4, 4])   # each booking's weekday, 0 = Monday
>>> counts = bookings.value_counts().sort_index()  # Monday to Friday only
>>> print(pd.concat([counts, pd.Series([0, 0], index=[5, 6])]).tolist())
[1, 2, 1, 1, 2, 0, 0]
>>> daily = pd.Series([31, 28, 33, 96, 30])        # day 3: a class visited
>>> print(daily.mean(), daily.drop(daily.idxmax()).mean())
43.6 30.5
```

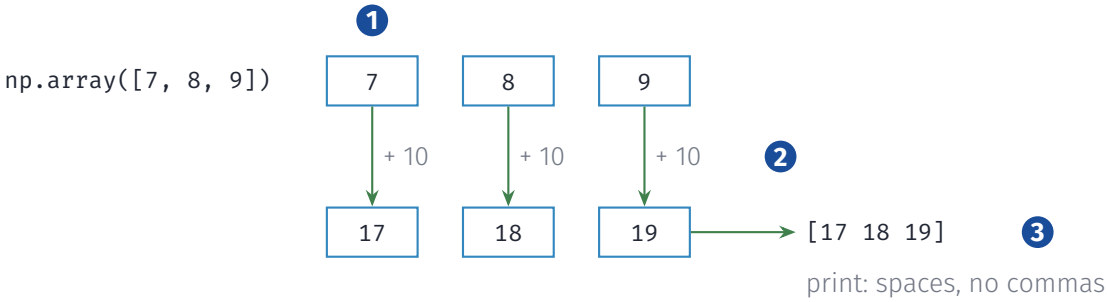
❶ A count sees only rows. The library is closed at the weekend, so its group bookings have no Saturday or Sunday rows, and `value_counts()` never names those days: a bar chart of the count passes for a full week. ❷ The honest week glues the closed days back on as zeros, `[1, 2, 1, 1, 2, 0, 0]`. ❸ One day a class came in, and its 96 visits among days of about 30 pull the mean up to 43.6. ❹ Without that day the mean is 30.5. A spike is dropped only for the question it distorts, and the chart or the sentence says so out loud.



Page 3 adds one question on week 6's arrays: what does `print(np.array([7, 8, 9]) + 10)` print? Walk it in its own order.

(1) `np.array([7, 8, 9])` turns the list into an array. (2) `+ 10` adds 10 to every element: numpy stretches the one number across the array and runs the sum as one compiled loop, with no Python step per element. (3) `print` shows the result the way numpy prints an array, with spaces and no commas:

```
[17 18 19]
```



A list refuses the same sum, since `+` on a list joins another list: `[7, 8, 9] + 10` stops with `TypeError: can only concatenate list (not "int") to list`.

Week 6's follow-up is on the Q&A page, under Week 6, Tuesday and Week 6, Friday.

1 · Paradigms, and why change working code

Slides, pages 8 to 11.

Paradigms: naming what we do

Page 8. A paradigm is a way of writing programs, and two programs written in different paradigms can do exactly the same thing. You have met several already. Machine code was the bottom rung of week 1's ladder. Procedural programming, sequences of procedures and functions, is the whole café so far. Functional programming, functions applied to data, and event-driven programming, routines triggered by events such as a button press, both arrive in week 8. Object-oriented programming writes a program as interacting objects that own their data and their behavior, and it is this week's.

The boundaries are fuzzy, and Python speaks every row but the machine's. Nothing gets thrown away: functions stay, and objects become a way to organize them.

OOP in one picture: two animals

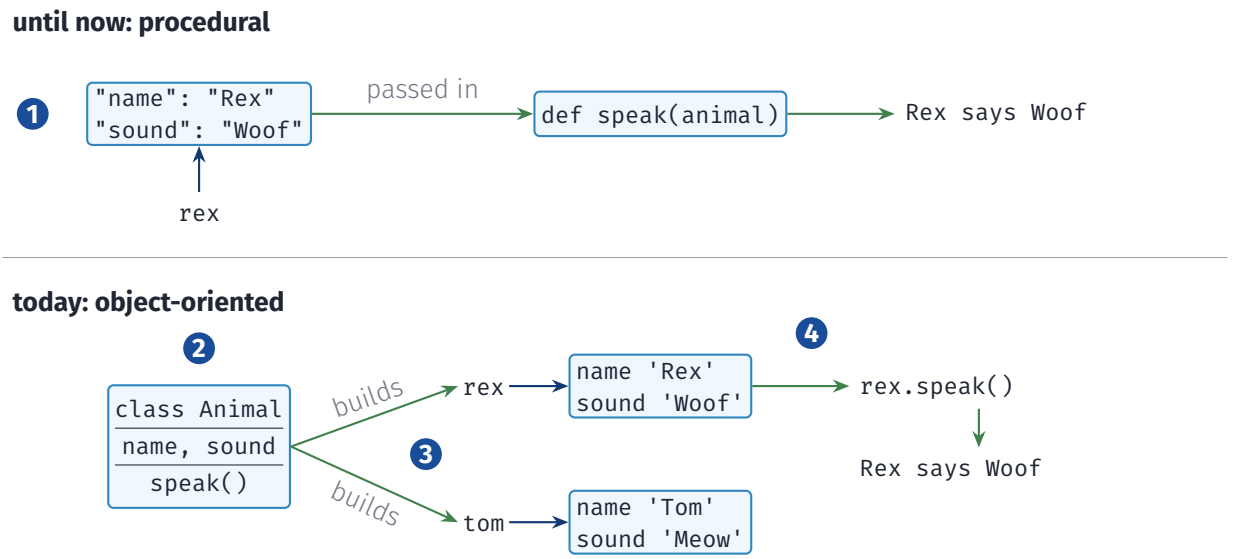
Page 9. The same two animals, written the way the café has been written all term, then as objects. Until now, each animal is a dictionary, and the behavior lives apart, in a function that takes the animal as an argument:

```
>>> rex = {"name": "Rex", "sound": "Woof"}
>>> def speak(animal):
...     return animal["name"] + " says " + animal["sound"]
...
>>> print(speak(rex))
Rex says Woof
```

Object-oriented, one class holds both the data and the behavior, and each object built from it carries its own data:

```
>>> class Animal:
...     def __init__(self, name, sound):
...         self.name = name
...         self.sound = sound
...     def speak(self):
...         return self.name + " says " + self.sound
...
>>> rex = Animal("Rex", "Woof")
>>> tom = Animal("Tom", "Meow")
>>> print(rex.speak())
Rex says Woof
>>> print(tom.speak())
Tom says Meow
```

1 Until now, `rex` is a dictionary of data, and the behavior lives apart, in `speak`: the data travels into the function as an argument, `speak(rex)`. 2 Object-oriented, one class, `Animal`, holds the data, a name and a sound, and the behavior, `speak`, together. 3 The class is the blueprint: it builds two objects, `rex` and `tom`, and each carries its own name and sound. 4 `rex.speak()` asks the object itself, and the answer is the same, `Rex says Woof`, in a new shape.



`speak` returns its text, so only `print` shows it from a program: a bare `rex.speak()` on a line of its own, in a file, shows nothing. The shell is the exception, since it echoes the value of every line it runs, quotes and all:

```
>>> rex.speak()
'Rex says Woof'
```

Section 2 explains every line of the class: `__init__`, `self`, and the call that builds an object.

The problem with a pile of functions

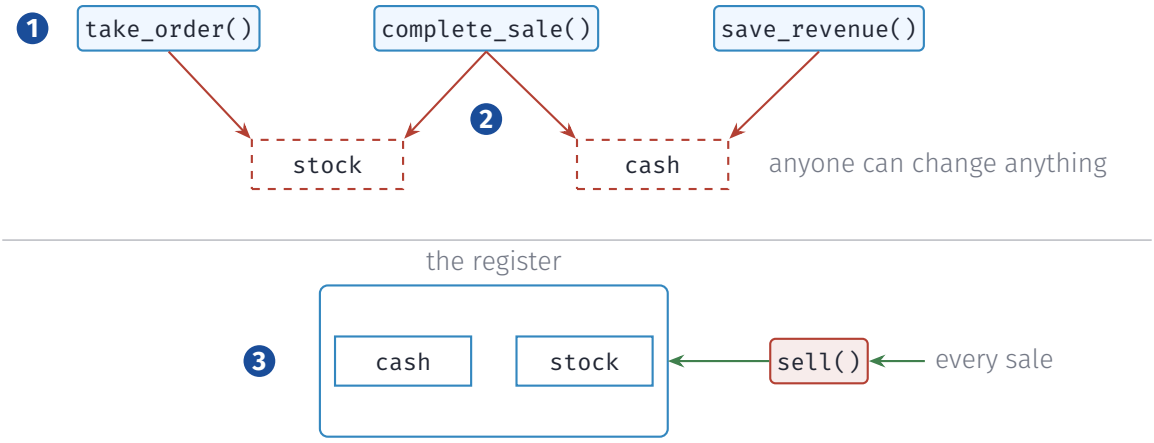
Page 10. Three things go wrong as the café grows, and each gets worse with size.

- **The money is exposed.** `revenue` is a plain number handed from function to function, and `stock` a plain dictionary any function can change. Nothing in the language makes a sale check the stock first.

- **Everything is passed by hand.** `complete_sale(order, stock, cash)` takes the money as an argument and hands it back changed. Forget to pass it, or to keep what comes back, and the books are quietly wrong.
- **The nouns are scattered.** An order and the register are real things in the problem, and the code has them only as loose variables.

What we want is easy to say and hard to get with functions alone: the register should be the only thing in the program that can touch the money.

1 In week 6's file, three functions reach the same two values, handed in by hand: `take_order` the stock, `complete_sale` the stock and the cash, and `save_revenue` the cash, which it writes to a file. 2 Nothing in the language says which of them may change what: the stock is a dictionary any of them can change, and the cash goes wherever it is passed. 3 The goal: one object, the register, keeps the cash and the stock inside it, and the only way to change them is its own method, `sell`.



The ask

Page 11. The owner's request, in a sentence: reorganize the café around the things in the problem, a menu item, an order, the register, so that the register alone owns the cash and the stock and decides which sales are allowed. That is a **refactor**: a change of structure that keeps what the program does. And a refactor is something you prove rather than claim: section 5 runs the old café and the new one on the same orders and compares the results.

The second beat decomposes the ask: underline its nouns, and they become the classes. A menu item, an order, the register and the café give `MenuItem`, `Order`, `Register` and `Cafe`. Week 6's notes closed on the first three: the things in the problem have names, and the code had none of them.

Common mistakes. Reading "object-oriented" as "no more functions": the methods of this week are functions defined inside a class, and the plain functions of weeks 2 to 6 keep their place. And calling any rewrite a refactor: a change that alters what the program computes is a new feature or a bug, whatever its shape.

Check yourself.

- **1.1** A refactor changes one thing about a program and must keep another. Which is which, and how do you show that the second one held?
- **1.2** Underline the nouns: "Reorganize the gym's booking script around its things: a member, a class session, and the front desk, which alone decides whether a booking goes through." Which classes does that give, and which one should own the bookings?

See also. py4e, chapter 14, the same ideas in another voice.

2 · Objects and classes

Slides, pages 12 to 16.

You have been using objects all term

Page 12. "Everything in Python is an object" has been true since week 1, and it can now be said precisely: an object is an instance of a class, and its class decides what it can do. `type()` has been naming the class all along:

```
>>> print(type(3), type("hi"))
<class 'int'> <class 'str'>
>>> l = [1, 2, 3]
>>> print(type(l), type(l[0]))
<class 'list'> <class 'int'>
>>> print(l.count(3))
1
>>> print(l.upper())
AttributeError: 'list' object has no attribute 'upper'
```

A list is an object that refers to other objects, here three `int`s. `count` is a method the list class provides, so `l.count(3)` answers 1; the list class has no `upper`, so the last line stops with an `AttributeError`. Strings can `upper()`, lists can `count()`: the class defines the menu of abilities. From here on we stop ordering off Python's menu and write our own.

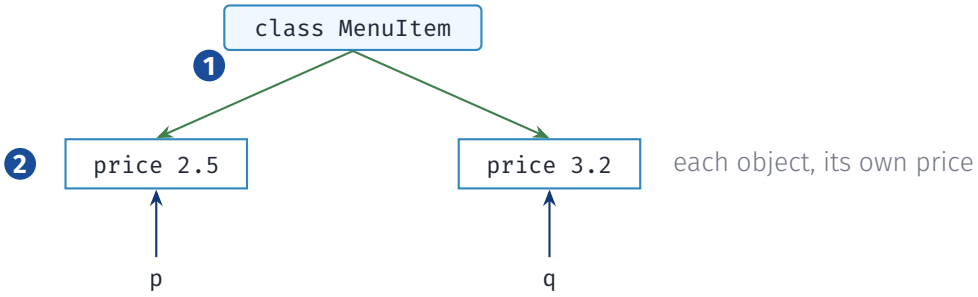
A class with nothing in it

Page 13. The smallest class there is holds only a docstring. Make two objects from it and bolt a price onto each by hand:

```
>>> class MenuItem:
...     """A menu item."""
...
>>> p = MenuItem()
>>> q = MenuItem()
>>> p.price = 2.50
>>> q.price = 3.20
>>> print(p.price, q.price)
2.5 3.2
>>> print(type(MenuItem), type(p), sep="\n")
<class 'type'>
<class '__main__.MenuItem'>
```

It works, and the last two lines hold a surprise: a class is itself an object, of type `type`, while `p` is a `MenuItem`. The `__main__.` in front of the name says the class was defined in the program you ran, here the shell, rather than imported from a file.

1 `MenuItem()` builds a new object from the class, a fresh one at each call, so `p` and `q` name two objects. 2 `p.price = 2.50` gives `p`'s object a price, and `q`'s object gets a price of its own. Each object carries its own attributes, and nothing makes sure every menu item gets one: forget one line and that item has no price at all.



A class is a specification, and each object is one thing built to it: one phone model, and every owner holds a phone of their own, with its own photos in it. Week 5's `id()` tells two objects apart:

```
>>> print(id(p) == id(q))
False
```

What we want is a method that runs by itself the moment an object is made and fills in what every object must have. That method is a **constructor**. Its rarely needed opposite, a **destructor**, runs when an object is deleted; the course never needs one.

`__init__`, `self`, and defaults

Page 14. In Python the constructor is spelled `__init__`, two underscores on each side:

```
class MenuItem:
    def __init__(self, name, price=0.0):   # runs automatically at creation
        self.name = name
        self.price = float(price)          # coerce: MenuItem("tea", "2.2") works too

    def __str__(self):                     # how print() shows it
        return f"{self.name} ({self.price:.2f})"
```

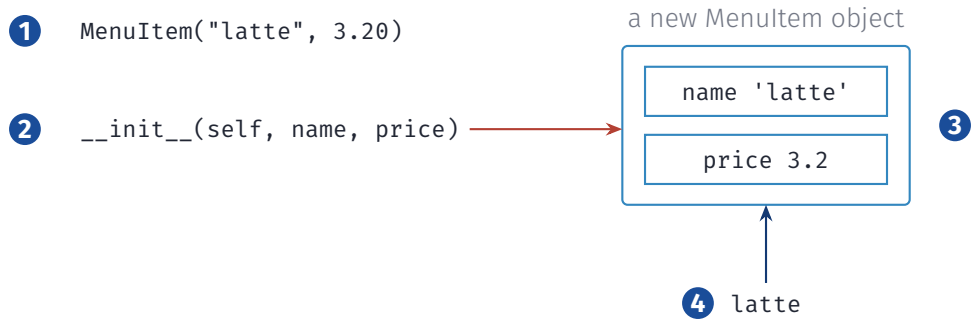
Three things to see. Every method's first parameter is `self`, this particular object: you never pass it, Python does. The constructor converts the price with `float`, so `MenuItem("tea", "2.2")` stores the number 2.2 and not the text, week 2's type discipline. And `__str__` is how `print` shows the object:

```
>>> latte = MenuItem("latte", 3.20)
>>> print(latte)
latte (3.20)
>>> print(latte.name, latte.price)
latte 3.2
>>> print(MenuItem("water").price, MenuItem("water", "2").price)
0.0 2.0
```

The default, `price=0.0`, makes `MenuItem("water")` legal, and the conversion turns the text `"2"` into `2.0`.

Why two underscores on each side? Not to make the method private. A name spelled that way, `__init__` or `__str__`, is one of Python's special methods, the ones Python calls by itself: `__init__` when an object is made, `__str__` when it is printed. They are public, and the spelling must be exact. Write `__int__` by mistake and Python never runs it, so `MenuItem("latte", 3.20)` stops with `TypeError: MenuItem() takes no arguments`. Private, in section 3, is two underscores at the front only.

1 `MenuItem("latte", 3.20)` first makes a new, empty `MenuItem` object. 2 Python then calls `__init__` with that object as `self`, and your two values as `name` and `price`. 3 `self.name = name` and `self.price = float(price)` store the values on that object. 4 The name `latte` points at the finished object. That is why `self` heads every method: it is how a method reaches the object it works on.



A value an object holds, such as its `name`, is an **attribute**; a function its class defines, such as `__str__`, is a **method**. Attributes belong to the objects, and methods belong to the class: two `MenuItem`s share the one copy of `__str__`'s code, and each has its own `name` and `price`. An attribute can also live on the class itself, shared by every object; the course never needs one. This `__str__` is the simple version, and section 4 gives it a `kind()`.

> **Good practice.** Create every attribute in `__init__`, even when its first value is a placeholder such as `0` or `None`. Every object of the class then has the same attributes from the moment it exists, and a reader finds them all in one place instead of hunting for the line that bolted one on later.

`self`, and the mistakes around it

Page 15. `print(latte)` calls `latte.__str__()` for you, which is the payoff of writing one. Two mistakes sit right beside it. The first is the most common beginner error in object-oriented code, leaving `self` out of a method's parameters:

```
>>> class Clock:
...     def __str__():
...         return "noon"
...
>>> print(Clock())
TypeError: Clock.__str__() takes 0 positional arguments but 1 was given
```

Python hands every method the object as its first argument, and a method with no parameter has no place to receive it, so the call fails the moment it happens. The second is scope: inside a method, `self.price` is the object's price and a bare `price` is a local name. A method other than `__init__` that reads a bare `price` stops with `NameError: name 'price' is not defined. Did you mean: 'self.price'?`: week 2's scope rules, now inside a class.

Poll. `class Box:` stores `self.n` in `__init__(self, n)`, and its method `total(self)` returns `self.n * 2`. What does `print(Box(5).total())` show: `10`, `5`, `25`, or an error, since `self` was never passed? It shows `10`: `Box(5)` stores 5 as `self.n`, and `total` doubles it. You never pass `self`; Python does, at every method call.

Your turn: a `Customer` class

Page 16. Write a class `Customer` whose `__init__(self, name, visits=0)` stores both values, with a `__str__` that returns text such as `"Ada: 4 visits"`. Then make `ada = Customer("Ada", 4)` and `print(ada)`. Copy the shape of `MenuItem`: the constructor stores each parameter on `self`, `__str__` returns an f-string built from them, and `print(ada)` calls it. Sheet 7's first exercise practices the same shape with a private attribute added, and its worked solutions are beside it on Brightspace.

Common mistakes. Leaving out `self`, as above. Forgetting the parentheses after the class: `latte = MenuItem` makes `latte` a second name for the class, not an object, and `print(latte)` shows `<class '__main__.MenuItem'>` where you expected a latte. And mixing the two ways of section 1: a dictionary's values are read by key, `rex["name"]`, and an object's by attribute, `rex.name`. Swap them and Python says which one it holds: `AttributeError: 'dict' object has no attribute 'name'`, or `TypeError: 'Animal' object is not subscriptable`.

Check yourself.

- **2.1** What does this print?

```python
class Lamp:
    def __init__(self, room, watts=40):
        self.room = room
        self.watts = watts

a = Lamp("hall")
b = Lamp("desk", 60)
a.watts = 25
print(a.watts, b.watts)
```

- **2.2** This method stops with a `TypeError` as soon as it is called. Why, and what is the fix?

```python
class Lamp:
    def __init__(self, room):
        self.room = room

    def switch_on():
        return "on"
```

See also. Sheet 7, exercise 1. The Function reference, Classes and objects. py4e, chapter 14. The official Python tutorial, chapter 9, "Classes".

3 · Encapsulation

Slides, pages 17 to 21.

The four pillars

Page 17. The rest of the week hangs on four ideas, and page 17 says to learn them as a set, since the exam can ask them by name:

1. **Encapsulation:** an object hides how it works and protects its own data.
2. **Inheritance:** classes can be organized in hierarchies and share code.
3. **Abstraction:** a base class can demand abilities without providing them.
4. **Polymorphism:** the same operation behaves differently on different objects.

They come in order, and the café gives each one a job: encapsulation in this section, inheritance in section 4, abstraction and polymorphism in section 6.

Encapsulation: hide the how, protect the what

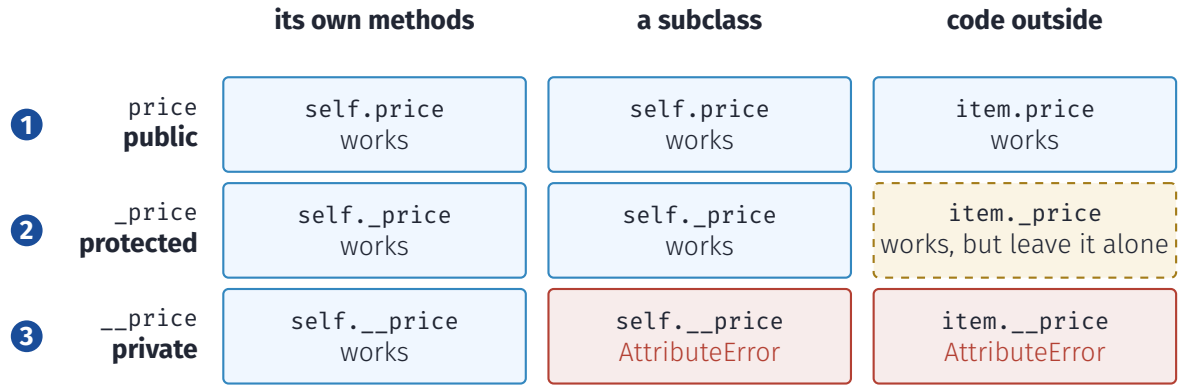
Page 18. You do not need to know how a register works out change in order to use one. A class offers a **public interface**, the methods others may call, and behind it the class is free to change how it works, as long as the interface keeps its promises. Python marks what is meant for outsiders by the name alone:

- **Two leading underscores, `__price`, make an attribute private.** Python *mangles* the name inside the class, so `self.__price` is stored as `_MenuItem__price`, and the plain name stops working from outside. It is a strong keep-out sign, not a lock: Python has no truly private data.
- **One leading underscore, `_price`, makes it protected, by convention only:** outsiders please stay away, and subclasses may use it (section 4).
- **Anything else is public.**

Public, protected, private: who may touch it

Page 19. Page 18's three spellings, read by three kinds of code: the class's own methods, the methods of a subclass, and code outside holding a `MenuItem` called `item`.

❶ Plain `price` is public: the class's own methods, a subclass's methods and code outside all reach it. ❷ `_price`, with one underscore, is protected, and it is only a sign. The class and its subclasses use it freely; code outside can too, since Python lets it through, but the sign says leave it alone. ❸ `__price`, with two, is private, and here Python acts: inside `MenuItem` it stores the attribute as `_MenuItem__price`, so `self.__price` works there, and the plain name misses everywhere else, a subclass included, with an `AttributeError`.



One underscore asks; two underscores rename. Section 4 shows why the subclass misses it, and the error message that gives it away.

The validating setter

Page 20. If `price` is a plain attribute, anyone can write `latte.price = -3`, and a latte now costs minus three euros. So the class takes the pen. This is `set_price`, as `cafe_w7.py` writes it:

```
    def set_price(self, new_price):
        """Change the price -- but never to a negative. The object protects itself."""
        new_price = float(new_price)     # coerce first ("3.5" is fine), THEN validate
        if new_price >= 0.0:
            self.price = new_price
```

A **setter** is a method that changes an attribute with the rules attached. It converts first, so the text `"3.5"` is fine, then checks: only a price of zero or more lands. Try it in the shell, started in the folder that holds `cafe_w7.py`, `cafe_w6b.py` and `register_sayings.py`, since the comparison with week 6 later on needs all three:

```
>>> from cafe_w7 import MenuItem
>>> m = MenuItem("latte", 3.20)
>>> m.set_price(-3)
>>> print(m.price)
3.2
>>> m.set_price("2.5")
>>> print(m.price)
2.5
```

The negative bounced off the check and the old price stayed; the text became a number. Ours keeps the old value on a bad input, and the textbook variant clamps it to `0.0` instead. Either way the object enforces the rule, not its caller. A **getter**, such as a `get_price`, is the matching method for reading a private attribute. The café keeps `price` itself public: the setter is the door with the rule, offered beside it.

The authoritative register, and the one-way door

Page 21. The register is the heart of the design. Its constructor stores the cash and the stock under private names, `__cash` and `__stock`, and one method reads the cash:

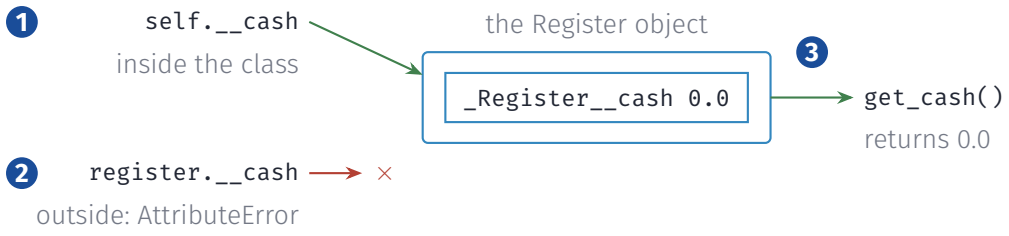
```
class Register:
    def __init__(self, cash, stock):
        self.__cash = cash          # name-mangled: outsiders cannot touch the money directly
        self.__stock = stock        # dict: item name -> units available

    def get_cash(self):
        return self.__cash
```

Try to reach the money directly:

```
>>> from cafe_w7 import Cafe
>>> register = Cafe(cash=0.0).register
>>> print(register.get_cash())
0.0
>>> print(register.__cash)
AttributeError: 'Register' object has no attribute '__cash'
>>> print(hasattr(register, "set_cash"))
False
```

❶ Inside the class, Python rewrites `self.__cash` as `self._Register__cash`, so the object keeps the money under the mangled name. ❷ Outside, nothing is rewritten: `register.__cash` asks for an attribute called `__cash`, the object has none, and the line stops with an `AttributeError`. ❸ `get_cash`, written inside the class, reads `_Register__cash` and hands back `0.0`: the one door. And `hasattr` confirms what is missing: there is no `set_cash`.



The money has one owner, and what is missing matters more than what is there. `MenuItem` has `set_price`, because prices really change. The register has no `set_cash`, because the drawer changes only through a sale. Encapsulation is more than hiding data: it is choosing which doors exist.

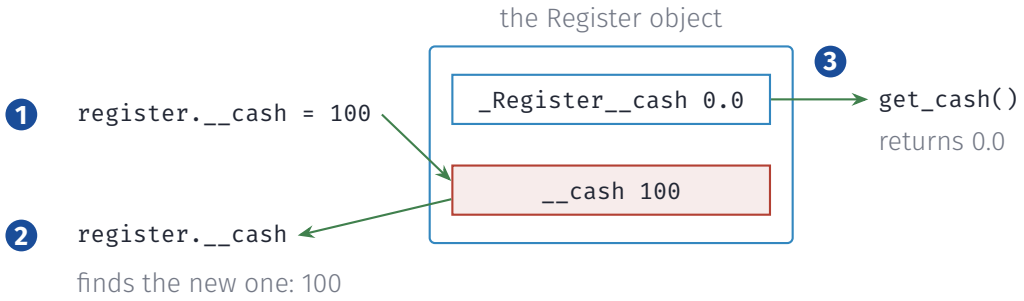
Good practice. A double underscore is a promise about who changes the data, not a lock. `register._Register__cash` still reads the money, as step ❶ shows, and code that writes it is reaching around the door on purpose. Treat a private name in someone else's class as closed: code that uses one skips every rule the class enforces, and it breaks the day the class renames its attribute.

Writing to a private name from outside

Reading `register.__cash` from outside fails, as page 21 shows. Writing to it does not: no error comes, and reading it back then seems to work.

```
>>> register.__cash = 100
>>> print(register.__cash)
100
>>> print(register.get_cash())
0.0
```

❶ Outside a class nothing is renamed, so `register.__cash = 100` creates a new attribute whose name happens to be `__cash`. ❷ Reading `register.__cash` now finds that new attribute, and its 100 looks like the money. ❸ `get_cash`, inside the class, still reads `_Register__cash`, and the drawer holds 0.0: the private attribute never moved.



The object now holds two attributes where you meant one, and nothing warns you. The rule for code outside a class is short: never write to a name that starts with two underscores.

In the project. The agent contract, project document 03, says in its parts "How an order is executed" and "The reply you get back" which object owns your balances and what your agent hears back. Read them beside page 21: the one-way door is the same question.

Common mistakes. Writing to the private name from outside, as above: no error, and the money does not move. Reading it from outside, `register.__cash`, before any such write, stops with page 21's `AttributeError`. And calling a private name protected: `__cash`, with two underscores, is private and renamed; `_cash`, with one, would be protected, only a sign, and code outside could still read it.

Check yourself.

- **3.1** `MenuItem` offers `set_price`, and `Register` offers `get_cash` but no `set_cash`. Why the difference?

- **3.2** What does this print?

```
class Tank:
    def __init__(self, liters):
        self.liters = liters

    def set_liters(self, value):
        value = float(value)
        if value >= 0.0:
            self.liters = value

t = Tank(40)
t.set_liters("-5")
t.set_liters("12")
print(t.liters)
```

See also. Sheet 7, exercise 1, its private count and its validating setter. The Function reference, Private and protected. py4e, chapter 14, which skips encapsulation: for it, the slides and these notes are the reference.

4 · Inheritance

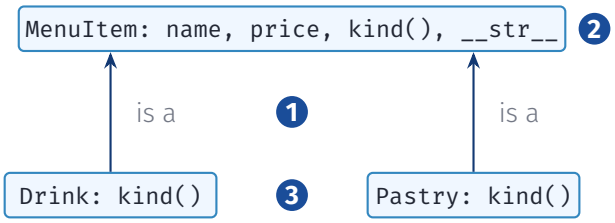
Slides, pages 22 to 26.

Hierarchies of things

Page 22. Real things fall into hierarchies. A drink and a pastry are both menu items: they share a name and a price, and they differ in kind. Inheritance lets classes share in exactly that way, and its vocabulary is examinable:

- `MenuItem` is the **base class**, also called the *super class* or the *parent class*;
- `Drink` and `Pastry` are **derived classes**, also called *subclasses* or *children*;
- a derived class **inherits** the base's attributes and methods, adds its own, and may **override** what it inherits.

(1) `Drink` and `Pastry` each point up at `MenuItem`: a drink is a menu item, and so is a pastry. (2) What `MenuItem` defines, `name`, `price` and `__str__`, is written once, and both subclasses inherit it. (3) Each subclass writes the one thing that differs, its own `kind()`. It is week 2's reason for functions: write the common part once.



Both designs are among this week's files. `w07_items_subclasses.py` builds the hierarchy, and `w07_items_category.py` is the road not taken: one `MenuItem` class with a `category` field. The second is simpler, and a new category needs only a new word; the first gives each kind its own class, which is what the rest of the week builds on.

Inheritance, in code

Page 23.

```
class MenuItem:
    def __init__(self, name, price=0.0):
        self.name = name
        self.price = float(price)
    def kind(self):
        raise NotImplementedError("every MenuItem subclass must define kind()")
    def __str__(self):
        return f"{self.name} ({self.kind()}, {self.price:.2f})"

class Drink(MenuItem):
    def __init__(self, name, price):
        MenuItem.__init__(self, name, price)
    def kind(self):
        return "drink"
```

The base `MenuItem` grows the piece section 2 left out: its `__str__` now calls `self.kind()`, so it reads name, kind and price. But the base refuses to say what kind it is, and its `kind` raises `NotImplementedError`. `Drink` inherits from it, written `class Drink(MenuItem):`; it provides its own `kind`, and its `__init__` overrides the base's and then calls it by name. `Pastry` is the same with `return "pastry"`.

Three rules to keep straight

Page 24.

- **Call the base constructor by name:** `MenuItem.__init__(self, name, price)`. The course uses this explicit form. You will meet `super().__init__(name, price)` in books, online and in code an AI tool writes: it means the same, so recognize it; the course does not ask for it.
- **Private stays private.** The base's `__attributes` are mangled with the base's own class name, so a subclass that writes the plain name misses them. Its `_protected` ones a subclass can use, which is why a base marks what its subclasses need with one underscore.
- **Same name wins.** Define a method the base already has, and yours overrides it, as `kind()` and `__init__` do in `Drink`.

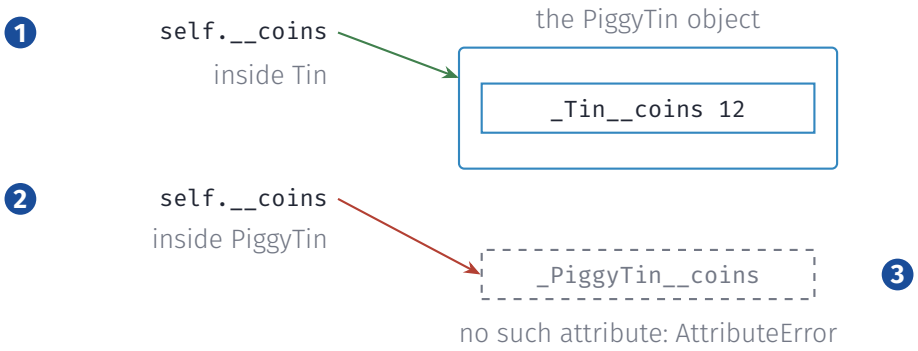
The private rule is the one people forget, and its error message holds the clue. A base `Tin` keeps its coins under a private name, and a subclass `PiggyTin` tries to read them:

```
>>> class Tin:
...     def __init__(self, coins):
...         self.__coins = coins
...
>>> class PiggyTin(Tin):
...     def peek(self):
...         return self.__coins
...
>>> piggy = PiggyTin(12)
>>> print(piggy._Tin__coins)
12
>>> piggy.peek()
AttributeError: 'PiggyTin' object has no attribute '_PiggyTin__coins'. Did you mean: '_Tin__coins'?
```

From Python 3.15 the last line ends with another wording of the same advice:

```
Did you mean '._Tin__coins' instead of '._PiggyTin__coins'?
```

(1) `PiggyTin(12)` runs the `__init__` it inherits from `Tin`, and inside `Tin` the name `self.__coins` is rewritten as `self._Tin__coins`: the 12 is stored there, as `piggy._Tin__coins` shows, a long name for looking, never for code. (2) Inside `PiggyTin`, the same spelling is rewritten with the subclass's own name, `self._PiggyTin__coins`. (3) The object has no attribute by that name, so `peek` stops with an `AttributeError`, and the suggestion names where the coins really are.



Python 3.15 words the suggestion as a choice between the two names, `._Tin__coins` instead of `._PiggyTin__coins`. The suggestion comes only when the two class names are close: had `MenuItem` stored `self.__price`, a `Drink` method reading it would stop at `'Drink' object has no attribute '_Drink__price'`, with no suggestion, and the subclass's name inside the missing name is the whole clue.

The trace: which `kind()` runs?

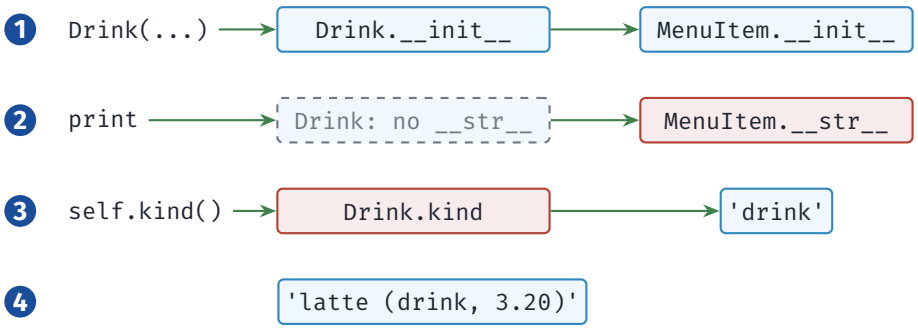
Page 25.

Poll. With page 23's classes, `print(Drink("latte", 3.20))`: which `kind()` runs, and what prints? The derived `Drink.kind`, printing `latte (drink, 3.20)`; the base class's `kind`; both, base first; or an error? The derived `Drink.kind`, and the line prints `latte (drink, 3.20)`: the object's own method wins.

```
>>> print(Drink("latte", 3.20))
latte (drink, 3.20)
```

Follow the calls by hand, in four steps, and the answer explains itself.

(1) `Drink("latte", 3.20)` runs `Drink.__init__`, which calls `MenuItem.__init__`, which sets `name` and `price`. (2) `print` asks the object for its `__str__`. `Drink` has none of its own, so Python looks one level up and finds `MenuItem.__str__`. (3) That `__str__` calls `self.kind()`, and the search starts again from the object's own class: `self` is a `Drink`, so `Drink.kind` runs and returns `"drink"`. (4) The f-string builds the text `'latte (drink, 3.20)'`, which `print` shows without its quotes. Step (3) is the point: `__str__` lives in the base, yet the object's own `kind()` was chosen. That choice has a name, polymorphism, the fourth pillar.



Page 43 lists exactly this among what the exam can ask: read a subclass and say which method runs.

Multiple inheritance

Page 26. A class may inherit from several bases at once:

```
>>> class A1:
...     def a1(self): return "A1"
...
>>> class A2:
...     def a2(self): return "A2"
...
>>> class B(A1, A2):
...     def b(self): return "B"
...
>>> class C(B):
...     def c(self): return "C"
...
>>> x = C()
>>> print(x.a1(), x.a2(), x.b(), x.c())
A1 A2 B C
```

`C` can do everything up its family tree. It is powerful and quickly confusing: know that it exists, since the slides call it examinable, and use it sparingly; the course never needs it.

Common mistakes. Overriding `__init__` and forgetting the base's: a `Drink` whose `__init__` stores nothing and never calls `MenuItem.__init__` builds an object with no `name`, and printing it stops with `AttributeError: 'Drink' object has no attribute 'name'`. And reading the base's private attribute from a subclass, the `PiggyTin` `AttributeError` above: give the base a getter, or mark the attribute with one underscore.

> **In the project.** The file you copied at the launch, `agent_stub.py` in the Project code and data folder, holds a class. The agent contract, project document 03, in its part "The file and the class", names the class and the method the competition calls: this week's vocabulary is the vocabulary of that file.

Check yourself.

- **4.1** What do the two lines print?

```
class Plant:
    def __init__(self, name):
        self.name = name

    def needs(self):
        return "water"

    def __str__(self):
        return self.name + " needs " + self.needs()

class Cactus(Plant):
    def needs(self):
        return "sun"

print(Plant("fern"))
print(Cactus("spike"))
```

- **4.2** Give `Cactus` an `__init__(self, name)` whose only line is `self.spines = True`. What happens at `print(Cactus("spike"))`, and why?

See also. Sheet 7, exercise 1. The Function reference, Inheritance. The official Python tutorial, chapter 9, for inheritance in depth.

5 · The refactor

Slides, pages 27 to 33.

Poll. Page 27, the break, leaves one question to turn over: for six weeks the money was a loose variable, so who should be allowed to change it? The register itself, through `sell()`. This section builds that register, and section 3's private `__cash` is how it keeps the money to itself.

Build the `Order` class

Page 28.

Poll. Type the `__init__` header for a `Drink` with a `name` and a `size`. The header puts `self` first, then the two values the caller passes:

```
def __init__(self, name, size):
```

An order is a list of item-and-quantity lines, and now it is an object with behavior:

```
class Order:
    def __init__(self):
        self.lines = []

    def add(self, item, quantity):
        self.lines.append((item, quantity))

    def total(self):
        total = 0.0
        for item, quantity in self.lines:
            total = total + item.price * quantity
        return total
```

`__init__` gives each order its own empty list, `self.lines`; `add` appends one `(item, quantity)` tuple; `total` adds up price times quantity. `total` is week 6's free-standing `order_total` function, moved next to the data it works on. That move, from a loose function to a method on the data, is most of what the refactor is. The file's version has a `__str__` too:

```
>>> from cafe_w7 import Order, Drink, Pastry
>>> order = Order()
>>> order.add(Drink("latte", 3.20), 2)
>>> order.add(Pastry("croissant", 2.80), 1)
>>> print(order.total())
9.2
>>> print(order)
Order of 2 line(s), total 9.20
```

`Register.sell`: validate, then mutate

Page 29. The method the term has walked toward, in two halves:

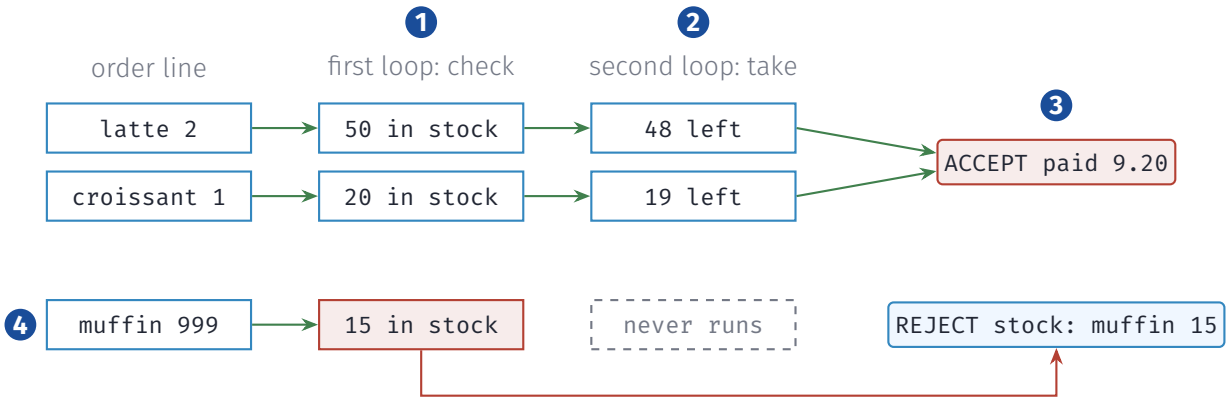
```
    def sell(self, order):
        # Validate the whole order before changing anything.
        for item, quantity in order.lines:
            have = self.__stock.get(item.name, 0)
            if have < quantity:
                return (False, "REJECT stock: " + item.name + " " + str(have))

        # Accepted: take the stock, apply the discount, take the money.
        for item, quantity in order.lines:
            self.__stock[item.name] = self.__stock[item.name] - quantity
        paid = order.total()
        if paid > DISCOUNT_THRESHOLD:
            paid = paid * (1.0 - DISCOUNT_RATE)
        self.__cash = self.__cash + paid
        return (True, f"ACCEPT paid {paid:.2f}")
```

The café starts with 50 lattes, 20 croissants and 15 muffins, among its six products. Run two orders through it:

```
>>> from cafe_w7 import Cafe
>>> cafe = Cafe(cash=0.0)
>>> print(cafe.serve([("latte", 2), ("croissant", 1)]))
(True, 'ACCEPT paid 9.20')
>>> print(cafe.register.get_stock("latte"), cafe.register.get_stock("croissant"))
48 19
>>> print(cafe.serve([("muffin", 999)]))
(False, 'REJECT stock: muffin 15')
>>> print(cafe.register.get_stock("muffin"), cafe.register.get_cash())
15 9.2
```

(1) The first loop only reads: every line of the order is checked against the stock, 2 lattes against 50 and 1 croissant against 20, and nothing moves. (2) Only when every line passes does the second loop run: it takes the stock down to 48 and 19, and the 9.20 goes into the drawer. (3) The reply is a pair, `(True, 'ACCEPT paid 9.20')`. (4) The muffin order fails the first loop, 999 against 15, so the second loop never runs: the reply is `(False, 'REJECT stock: muffin 15')`, and the stock is still 15. Validate before you mutate: week 5's rule, first felt in week 3's muffin rush, is now the shape of the method, and a rejected sale changes nothing.



The shape it hands back

Page 30. `sell` answers every order with a pair, `(accepted, message)`, and page 30 shows both answers in a session beside `cafe_w7.py`, on a register that holds 50 lattes:

```
>>> from cafe_w7 import Register, Order, Drink
>>> register = Register(0.0, {"latte": 50})
>>> five, sixty = Order(), Order()
>>> five.add(Drink("latte", 3.20), 5)
>>> sixty.add(Drink("latte", 3.20), 60)
>>> print(register.sell(five))
(True, 'ACCEPT paid 14.40')
>>> accepted, message = register.sell(sixty)
>>> print(accepted, message)
False REJECT stock: latte 45
>>> print(register.get_stock("latte"))
45
>>> print(register.get_cash())
14.4
```

Five lattes are in stock, and their 16.00 is over the 10.0 threshold, so 10% comes off: the reply is `(True, 'ACCEPT paid 14.40')`. Sixty lattes meet the 45 left, so the first loop refuses, and the

caller unpacks the pair into two names, `accepted` and `message`. The refusal moved nothing: still 45 lattes, still 14.4 in cash.

Week 5 asked: raise, or reply? The café kept reply, the road the register's heart was to be built on. A raise would stop the program unless every caller caught it with `try` and `except`; the reply is data the caller reads before carrying on. And, as page 30 says, peek at your brief: your project's ACCEPT and REJECT protocol answers the same way.

One hole, for the curious, as page 30 notes: the check trusts each line on its own, so an order that lists one product on two lines can slip past it. On the same register, with 45 lattes left:

```
>>> hole = Order()
>>> hole.add(Drink("latte", 3.20), 30)
>>> hole.add(Drink("latte", 3.20), 30)
>>> print(register.sell(hole))
(True, 'ACCEPT paid 172.80')
>>> print(register.get_stock("latte"))
-15
```

Each line of 30 passes the check against 45, and the stock ends at minus 15. Real registers add the lines up first; hunting holes like this one is week 10's whole job.

One more thing `sell` never asks: whether it has seen an order before. An order is an object, and it stays after its sale, so selling the same `five` twice, on a fresh register, sells ten lattes and takes the money twice:

```
>>> register = Register(0.0, {"latte": 50})
>>> five = Order()
>>> five.add(Drink("latte", 3.20), 5)
>>> print(register.sell(five))
(True, 'ACCEPT paid 14.40')
>>> print(register.sell(five))
(True, 'ACCEPT paid 14.40')
>>> print(register.get_stock("latte"), register.get_cash())
40 28.8
```

The register checks the stock and nothing else: handing each order over once is its caller's job.

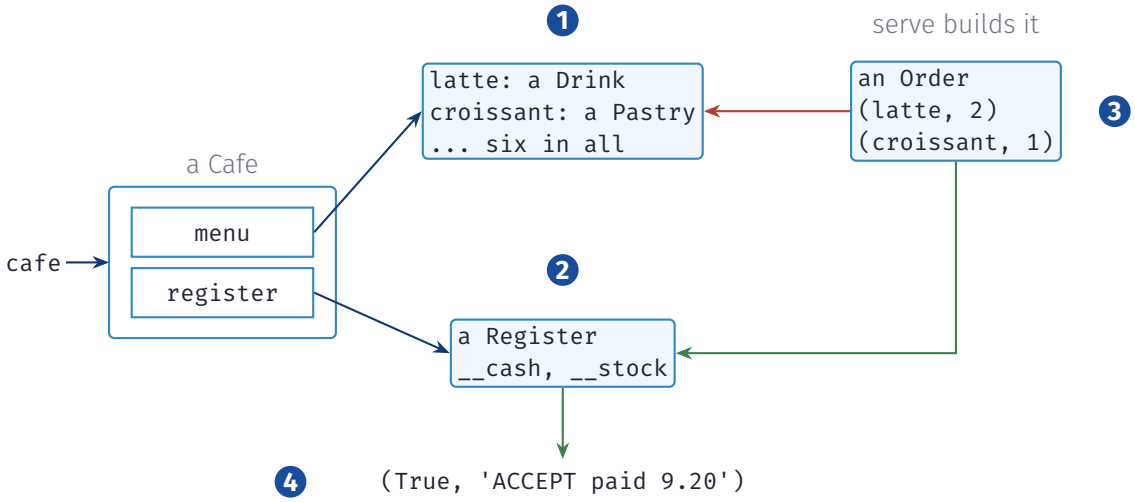
The `Cafe` ties it together

Page 31. The last class is the thin layer around the others:

```
class Cafe:
    def __init__(self, cash):
        self.menu = {"latte": Drink("latte", 3.20), ...}
        self.register = Register(cash, {"latte": 50, ...})

    def serve(self, requests):
        for name, quantity in requests:
            if name not in self.menu:                  # reject unknown item, WHOLE order
                return (False, "REJECT unknown item: " + name)
        order = self.build_order(requests)
        return self.register.sell(order)
```

1 Its `__init__` builds the menu, a dictionary of six real `Drink` and `Pastry` objects. 2 It gives the café its own `Register`, loaded with the starting stock: 50 espressos, 50 lattes, 40 cappuccinos, 20 croissants, 15 muffins and 30 teas. 3 `serve` keeps one guard of its own: an order that names anything off the menu is refused whole, as week 6's `complete_sale` refused what the café does not sell. Otherwise it builds an `Order` with `build_order`, whose lines point at the menu's own objects. 4 It hands the order to `register.sell`, which answers with the pair, `(True, 'ACCEPT paid 9.20')` for two lattes and a croissant. A café *has* a menu and a register, where a drink *is* a menu item: that is composition, beside section 4's inheritance.



The whole file runs as a demo, with no input. From the folder that holds it, `python3 cafe_w7.py` prints:

```
The Daily Grind (OOP edition). Menu:
  espresso (drink, 2.50)
  latte (drink, 3.20)
  cappuccino (drink, 3.00)
  tea (drink, 2.20)
  croissant (pastry, 2.80)
  muffin (pastry, 2.40)

Order 1 -> ACCEPT paid 9.20
Order 2 -> REJECT stock: muffin 15
Order 3 -> REJECT unknown item: wine

Cash in the register: 9.20
Muffins left: 15
```

One sale accepted at 9.20, two lattes at 3.20 and a croissant at 2.80, under the discount threshold; one refused, since 15 muffins cannot cover 999; one refused because the café does not sell wine. Every menu line printed itself through `MenuItem.__str__`, which called each item's own `kind`.

The refactor, side by side

Page 32. Read the change row by row:

Procedural (before)	Object-oriented (after)
`cash` and `stock` are loose variables	`Register` owns `__cash` and `__stock`
`complete_sale(order, stock, cash)`	`register.sell(order)`
any function can change `cash`	only `sell()` can
the nouns are variables	the nouns are classes
`order_total(order)`, a function	`order.total()`, a method

Every left cell is something loose, and every right cell is the same thing, owned. The same orders give the same money and the same stock, and the money now has one trustworthy door. One thing changed besides the shape, on purpose: the reply's wording. Week 6's register answered in plain words; week 7's answers ACCEPT or REJECT, the shape of page 30.

```
>>> import cafe_w6b
>>> stock = {"espresso": 50, "latte": 50, "cappuccino": 40,
...          "croissant": 20, "muffin": 15, "tea": 30}
>>> print(cafe_w6b.complete_sale([("wine", 1)], stock, 0.0))
(False, "we don't sell wine", 0.0)
>>> print(cafe.serve([("wine", 1)]))
(False, 'REJECT unknown item: wine')
```

Prove it: run both registers

Page 33. A refactor that changes the results is a bug, not a refactor, so the claim gets a test. `check_refactor.py` runs one list of orders through week 6's `complete_sale` and through week 7's café, then compares two things:

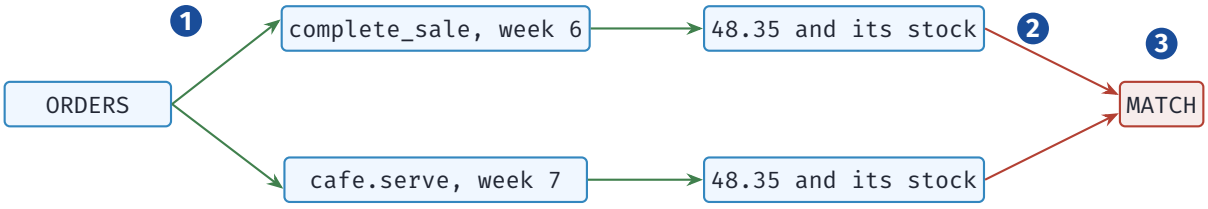
```
# check_refactor.py: runs the same orders through both versions
for order in ORDERS:
    _, _, proc_cash = proc.complete_sale(order, proc_stock, proc_cash)   # procedural
for order in ORDERS:
    cafe.serve(order)                                                    # OOP
oop_stock = {name: cafe.register.get_stock(name) for name in START_STOCK}
assert round(proc_cash, 2) == round(cafe.register.get_cash(), 2)
assert proc_stock == oop_stock
```

`complete_sale` hands back three values, and the procedural line keeps only the cash: `_` is an ordinary name that, by convention, holds a value nobody reads. Python keeps that value as it keeps any other, so the underscore saves no memory; it tells the reader. One name for all three, `result = complete_sale(...)`, would hold them together as a tuple. `oop_stock` reads the register's stock back through `get_stock`, one product at a time, with week 4's dict comprehension.

Run it from the folder that holds `cafe_w6b.py`, `cafe_w7.py` and `register_sayings.py`, with `python3 check_refactor.py`:

```
MATCH: revenue = 48.35
MATCH: stock   = {'espresso': 47, 'latte': 43, 'cappuccino': 38, 'croissant': 19, 'muffin': 11, 'tea': 28}
The refactor preserved behavior. Good.
```

(1) The same orders go through both registers, each from the same starting stock. (2) The two asserts compare what a sale changes: the revenue, rounded to the cent, and the leftover stock, product by product. (3) Any difference stops the program with an `AssertionError`, and that crash is the answer; when both hold, the MATCH lines print, 48.35 on both sides. The proof covers what it compares. The money and the stock are proved the same; the replies' wording is not compared, and it did change, on purpose.



`check_refactor.py` is week 5's unit-test idea at program scale, and week 3's habit of testing after every version, made automatic. We do not claim that the two registers agree; we assert it.

Good practice. Write the check before you touch the code: run the old version on fixed inputs, keep what it produced, and assert that the new version produces the same. Assert the values you observed, never the ones you expect, and run the check after every change. A check that runs in a second gets run; one that needs setting up does not.

In the project. The agent contract, project document 03, comes with `agent_unittest.py`, the same kind of check for your agent; its part "Check before you submit" says how to run it. Run it the way page 33 runs `check_refactor.py`: after every change.

Common mistakes. Forgetting a `return`: a `sell` whose accepting path ends without one hands back `None`, and `accepted, message = cafe.serve(...)` stops with `TypeError: cannot unpack non-iterable NoneType object`. Leaving out the parentheses: `paid = order.total` makes `paid` the method itself, and the discount test stops with `TypeError: '>' not supported between instances of 'method' and 'float'`. And changing anything before the whole order is checked: a refused sale that has already taken stock leaves the books wrong, week 3's muffin rush again.

Check yourself.

- **5.1** A register holds 4 scones. Three orders come in, one after the other: 3 scones, then 2 scones, then 1 scone. Which does `sell` accept, and how many scones are left after each?
- **5.2** `check_refactor.py` passes. Does that prove week 6's café and week 7's give the same replies? Why or why not?

See also. Sheet 7, exercise 2, the same refactor and proof on a smaller program. The Function reference, Two spellings, for the _ above.

6 · Abstraction and polymorphism

Slides, pages 34 to 37.

Pillar 3: abstraction

Page 34. Some ideas are too general to exist on their own. A vehicle is the idea behind a car and an airplane; a menu item is the idea behind a drink and a pastry. An **abstract class** defines the shared shape and demands abilities it refuses to provide, our `kind()`, whose body is `raise NotImplementedError("every MenuItem subclass must define kind()")`. Abstraction is how a base class writes a contract for its children.

The textbook says an abstract class cannot be instantiated. With our `raise` pattern it can, and the error arrives only when the missing method is called:

```
>>> from cafe_w7 import MenuItem
>>> water = MenuItem("water")
>>> print(water.name, water.price)
water 0.0
>>> print(water)
NotImplementedError: every MenuItem subclass must define kind()
```

Building the bare `MenuItem` works; printing it calls `__str__`, which calls `kind()`, which raises. The strict term for a method every subclass must provide is a *pure virtual function*. The standard library's `abc` module enforces the ban and refuses to build such an object at all; the `raise` pattern is the course's substitute, and its convention.

The contract is also what makes a new kind cheap. The day the café sells postcards, one class meets the demand, and `MenuItem` does not change:

```
>>> class Postcard(MenuItem):
...     def kind(self):
...         return "postcard"
...
>>> print(Postcard("harbor view", 1.50))
harbor view (postcard, 1.50)
```

`Postcard` writes only `kind`. It inherits `__init__` and `__str__` from `MenuItem`, and the inherited `__str__` finds the new `kind` the way section 4's trace found `Drink`'s.

Abstraction, coded from scratch

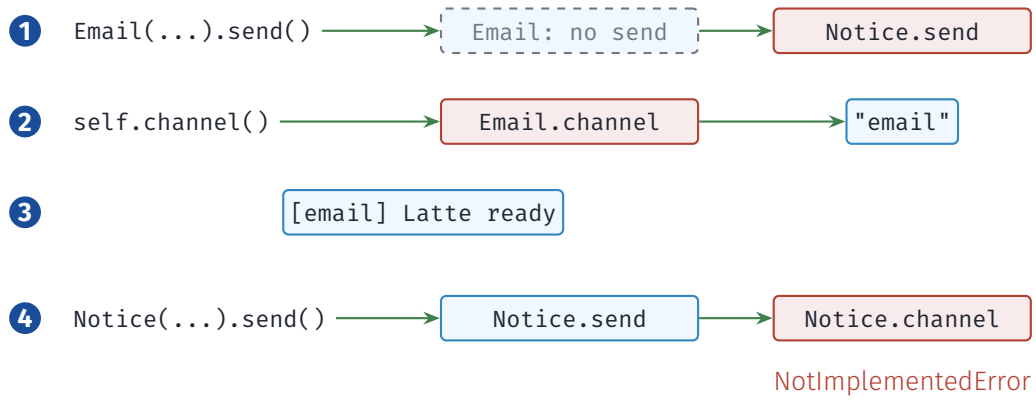
Page 35. Writing a contract yourself takes a base class that shares one method and demands another. On the café's notices to customers:

```
class Notice:
    def __init__(self, text):
        self.text = text
    def channel(self):
        raise NotImplementedError("define channel()")
    def send(self):
        return f"[{self.channel()}] {self.text}"

class Email(Notice):
    def channel(self):
        return "email"
```

```
>>> print(Email("Latte ready").send())
[email] Latte ready
>>> note = Notice("Latte ready")
>>> note.send()
NotImplementedError: define channel()
```

1 `Email("Latte ready").send()` looks for `send` in `Email`, finds none, and takes the one `Notice` shares, written once. 2 `send` calls `self.channel()`, and the search starts again at the object's own class: `Email` supplies `channel`, which returns `"email"`. 3 `send` builds `[email] Latte ready`. 4 On a bare `Notice`, the same search finds only the base's `channel`, the demand, which raises `NotImplementedError: define channel()`. The object builds, as the bare `MenuItem` did, and only the call stops.



It is the shape of the café's own `MenuItem`: `kind()` demanded, `__str__` sharing it, and `Drink` and `Pastry` supplying it.

Pillar 4: polymorphism

Page 36. You have used polymorphism since week 1 without the name:

```
>>> print(3 + 5)
8
>>> print("3" + "5")
35
>>> print([3] + [5])
[3, 5]
>>> for s in ("hello", [1, 2, 3], {"mary": 1234}):
...     print(len(s))
...
5
3
1
```

`+` adds two integers, joins two strings and joins two lists: one operator, three behaviors, chosen by the class of the operands. `len` works on a string, a list and a dictionary alike. The same operation, with a different behavior for each class: that is the definition.

`print` rests on the same idea. It calls `__str__` on whatever it is given, and `str(x)` is `x.__str__()`, so with `x = 3` and `y = 4.0` this sum is a classic trace:

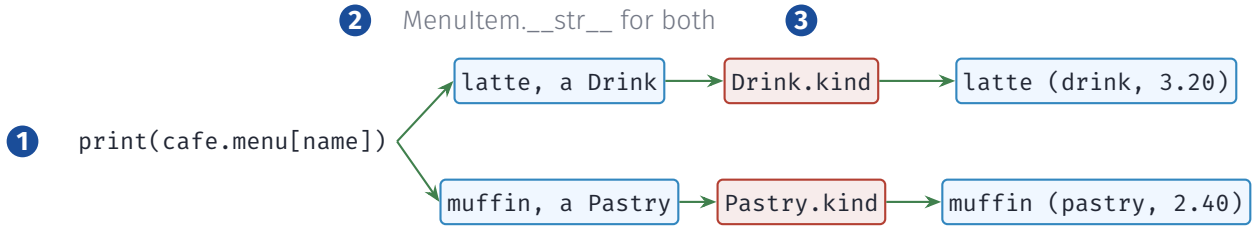
```
>>> x = 3
>>> y = 4.0
>>> print(x.__str__() + y.__str__())
34.0
```

The two texts `'3'` and `'4.0'` joined, not the number 7.0. A class with no `__str__` of its own falls back to a default that names the class and an address, such as `<__main__.Item object at 0x1029b6f28>`, with an address that changes from run to run. That is why the café's menu items and its orders define a `__str__`. The `Register` and the `Cafe` keep the default: `print(cafe.register)` shows only `<cafe_w7.Register object at 0x...>`.

The menu loop is where the pillar pays off:

```
>>> for name in cafe.menu:
...     print(cafe.menu[name])
...
espresso (drink, 2.50)
latte (drink, 3.20)
cappuccino (drink, 3.00)
tea (drink, 2.20)
croissant (pastry, 2.80)
muffin (pastry, 2.40)
```

(1) The loop makes the same call for every item and never asks which class it holds. (2) Each call runs the one `__str__`, written once in `MenuItem`, and it calls `self.kind()`. (3) The search for `kind` starts at each object's own class, so a drink answers `drink` and a pastry answers `pastry`. Inheritance and abstraction are how the café gets polymorphism, and encapsulation keeps the state safe while they work: the four pillars are one machine, not four separate tricks.



Good practice. Give every class you will ever print a `__str__`, early, even a rough one. The default, a class name and an address, tells you nothing while you debug; once the class has a `__str__`, every `print` of its objects, in every loop and every message you build, becomes readable.

Polymorphism, coded from scratch

Page 37. Building the pillar yourself takes three steps, and the loop at the end never asks which class it holds. On café payments:

```
class Payment:
    def __init__(self, amount):
        self.amount = amount
    def receipt(self):
        raise NotImplementedError

class Cash(Payment):
    def receipt(self):
        return f"paid {self.amount:.2f} in cash"

class Card(Payment):
    def receipt(self):
        return f"paid {self.amount:.2f} by card"
```

```
>>> for payment in [Cash(9.20), Card(14.40)]:
...     print(payment.receipt())
...
paid 9.20 in cash
paid 14.40 by card
```

The base class names the method and refuses to write it, the contract of pillar 3. Each subclass writes its own `receipt`, and inherits `__init__` from `Payment` untouched. The loop then makes one call, and Python looks `receipt` up in each object's own class, so the loop never asks whether it holds cash or a card. A new way to pay costs one class: a `Voucher` with its own `receipt` goes through this loop unchanged, and one without it stops at the base's `raise` the first time it is asked for a receipt.

Common mistakes. A `__str__` that prints instead of returning: the text appears once, then Python stops with `TypeError: __str__ returned non-string (type NoneType)`, since a method with no `return` returns `None`. Python 3.15 names the class, as in `TypeError: Item.__str__() must return a str, not NoneType`. And printing an object of the abstract base, as `print(water)` shows above.

Check yourself.

- **6.1** What does `print((2).__str__() + (1.5).__str__())` show, and why is it not `3.5`?
- **6.2** A base class `Sensor` demands a method of its subclasses, the course's way: its `read(self)` raises `NotImplementedError`, with the message "every Sensor subclass must define read()". What happens at `s = Sensor()`, and what at `s.read()`?

See also. The official Python tutorial, chapter 9, "Classes". py4e, chapter 14, which skips polymorphism.

7 · The pattern your project uses

Slides, page 38.

Section 3's register is more than a tidier café. It is a pattern, and one sentence holds it: one authoritative object owns the state, and everyone else goes through its accept or reject method. In the café the `Register` owns the cash and the stock. `sell` is the only way either one changes, and it answers every order with a pair, `(True, "ACCEPT paid ...")` or `(False, "REJECT ...")`. Nothing outside the class can move the money, because there is no `set_cash` to call: the drawer moves only through a sale that passed every check.

Page 38 maps the café's half of that sentence onto the project. Learn the pattern here, in the café, then apply it to the agent contract yourself: the slide calls that mapping the graded skill, and the course never shows the trading code.

> **In the project.** The brief, project document 01, in its section 3 on keeping the agent apart from the interface, describes the project's side. Read it with page 38 beside it, and find the object that owns the balances and the one method everything goes through.

Common mistakes. Giving the owner a second door: a `set_cash` added "just for testing" becomes the way other code changes the money, and the checks in `sell` stop protecting anything. And answering a refusal with a crash where the caller expects a reply: the pattern works only when every caller can read the answer and carry on.

Check yourself.

- **7.1** A `Ferry` keeps `self.__seats_left`, with a method `board(group)` that answers with a pair, `(accepted, message)`, and a method `get_seats_left()`; no other method touches `__seats_left`. Which code may change the seats left, and how does a caller learn whether a group got on?
- **7.2** Week 5 asked: raise, or reply? Which does `sell` do when an order is refused, and what does its caller do with the answer?

See also. Sheet 7, exercise 2: its `record` answers with a pair, as `sell` does.

8 · Errors, your turn, and Python against C++

Slides, pages 39 to 41.

Common OOP mistakes

Page 39. Four bugs that classes invite, and where each one hides:

```
class Order:
    def total():                        # 1. forgot self -> TypeError when called
        ...
    def __init__(self, lines=[]):       # 2. mutable default! all Orders share ONE list
        self.lines = lines

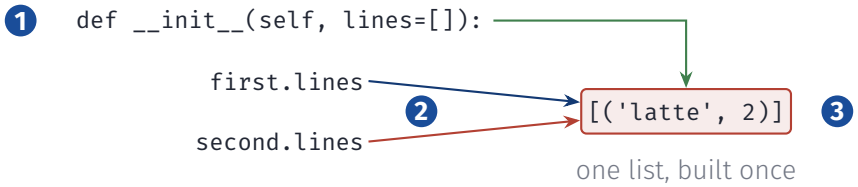
order.total                             # 3. no () -> the method object, not its result
register.__cash                         # 4. name-mangled -> AttributeError from outside
```

1. Forgetting `self`, section 2's error: calling the method stops with `TypeError: Order.total() takes 0 positional arguments but 1 was given`.
2. The mutable default, the subtle one, comes next.
3. `order.total` without parentheses is the method itself, not its result: `print(order.total)` shows `<bound method Order.total of <cafe_w7.Order object at 0x...>>`, and no total is ever computed.
4. `register.__cash` from outside: name mangling doing its job, section 3's `AttributeError`.

The mutable default deserves a run of its own. Try a small class with the same shape:

```
>>> class Tab:
...     def __init__(self, lines=[]):
...         self.lines = lines
...     def add(self, item, quantity):
...         self.lines.append((item, quantity))
...
>>> first = Tab()
>>> second = Tab()
>>> first.add("latte", 2)
>>> print(second.lines)
[('latte', 2)]
```

❶ Python builds the default `[]` once, when it reads the `def` line, not each time `__init__` runs. ❷ Every object made without a list of its own gets that same list, so `first.lines` and `second.lines` are two names for one object. ❸ A change through one object shows through all of them: week 5's aliasing bug, in a new place. The fix is to write `self.lines = []` inside `__init__`, so each object gets its own.



Good practice. When a parameter needs a list or a dictionary as its default, use `None` and build the fresh one inside:

```
def __init__(self, lines=None):
    if lines is None:
        lines = []
    self.lines = lines
```

Every object then gets a list of its own unless the caller hands one in. Week 6's sales report tests its `save_path` with the same `is None`.

Your turn: `restock`

Page 40. Add a method to `Register`:

```
def restock(self, name, quantity):
    """Add `quantity` units of `name` back into stock."""
    ...
```

Then the design question: should `restock` be a method on `Register`, or a plain function outside it? It has to be a method. Only code inside the class can reach `__stock`, since the name is mangled, so the code that changes the stock must live with the stock: encapsulation doing its job. The bonus: should `restock` accept a negative quantity? No, for the reason the validating setter refuses a negative price. One way to write it, inside the `Register` class:

```
    def restock(self, name, quantity):
        """Add `quantity` units of `name` back into stock."""
        if quantity > 0:
            self.__stock[name] = self.__stock.get(name, 0) + quantity
```

Then use it, the way the owner would when a delivery comes in, from a shell started in the folder that holds `cafe_w7.py`, which carries the method below `sell`:

```
>>> from cafe_w7 import Cafe
>>> register = Cafe(cash=0.0).register
>>> register.restock("muffin", 5)
>>> print(register.get_stock("muffin"))
20
>>> register.restock("muffin", -50)
>>> print(register.get_stock("muffin"))
20
```

The café opens with 15 muffins, so the delivery of 5 makes 20. One call on the register is the whole use: the caller names the item and the quantity, and never touches `__stock`. The -50 bounces off the guard and the stock stays at 20.

A name the café has never stocked is new to the stock, not to the menu:

```
>>> cafe = Cafe(cash=0.0)
>>> print(cafe.register.get_stock("postcard"))
0
>>> cafe.register.restock("postcard", 5)
>>> print(cafe.register.get_stock("postcard"))
5
>>> print(cafe.serve([("postcard", 1)]))
(False, 'REJECT unknown item: postcard')
```

`get_stock` answers 0 for a name it does not hold, since it reads the dictionary with `get(name, 0)`, and `restock` adds the name as a new key. That is all it adds: the menu has no postcard, so `serve` still refuses the order. Selling postcards takes a `Postcard` class, as section 6 writes one, and a place on the café's menu.

Bonus: the same class in Python and C++

Page 41. Week 1 showed Hello World in four languages, and here is the object-oriented edition, one `Person` class in both:

```
class Person:
    def __init__(self, name=""):
        self._name = name
    def get_name(self):
        return self._name
```

```
class Person {
  public:
    Person(std::string nname = "") {
      name = nname; }
    std::string get_name() { return name; }
  protected:
    std::string name;
};
```

The design is the same. C++ declares `public:`, `protected:` and `private:` with keywords and braces; Python says the same with naming conventions, `_name` and `__name`, and with indentation. The concepts transfer to any object-oriented language you meet next, which is the reason to learn them by name.

Common mistakes. Calling a method on the class instead of an object: `Order.total()` stops with `TypeError: Order.total() missing 1 required positional argument: 'self'`, since no object was there to be `self`. Calling a value: `latte.price()` stops with `TypeError: 'float' object is not callable`, because `price` is an attribute, not a method. And calling the module instead of its class: after `import cafe_w7`, the line `cafe_w7(cash=0.0)` stops with `TypeError: 'module' object is not callable`. The class inside is `cafe_w7.Cafe`, and `from cafe_w7 import Cafe` makes it plain `Cafe`.

Check yourself.

- **8.1** What does this print?

```
class Shelf:
    def __init__(self, books=[]):
        self.books = books

a = Shelf()
b = Shelf(["atlas"])
a.books.append("novel")
c = Shelf()
print(len(c.books), len(b.books))
```

- **8.2** Outside code reads `p._name` from page 41's Python `Person`. What happens, and what would two underscores, `self.__name`, change?

See also. Sheet 7, exercise 2. LPTHW, exercises 40 to 44, if you own it.

Where the café is now

Slides, pages 42 to 45.

The Daily Grind at the end of day 8: the sale week 6's `complete_sale` made now lives in `Register.sell`, in `cafe_w7.py`, and `check_refactor.py` proves that it moves the same money and the same stock. The text menu, options [1] to [8], and the reports stay in `cafe_w6b.py`. The `Register` is the single owner of the cash and the stock; `MenuItem` offers a validating setter for its price, which itself stays public; the menu items and the orders print themselves through `__str__`. You can name the four pillars and point at the café line that carries each: encapsulation in `self.__cash`, with `get_cash` and no `set_cash`; inheritance in `class Drink(MenuItem):`; abstraction in a `kind()` that raises `NotImplementedError`; polymorphism in a menu loop where every item prints its own kind. Week 8 puts functional tools, comprehensions, `map` and `lambda`, to work on the café's reports, and builds a first tkinter window on top of this same `Register`.

What the exam can ask from these slides, as page 43 lists it: write a small class from a description, with `__init__`, a method and `__str__`; name and define the four pillars; read a subclass and

say which method runs, the trace of section 4; explain `__private` against `_protected`; spot the forgot-`self` and the mutable-default bugs. Of the five, the trace is the one to drill.

Go through the checklist on page 45. Anything you cannot tick, read its section here again, then work sheet 7, whose worked solutions are on Brightspace beside it: the bank holiday removes its Monday session, and the week-9 practical takes questions on it. That session, Monday 2 November, is also check-in #1, where your group shows the refactor milestone, week 8's: come with running code. Week 7's own project milestone is the procedural one, a buy-and-hold back-test that runs, and the brief puts the class refactor in week 8. Self-check 7 is open on Brightspace until Thursday 29 October, at 23:59.

For reading, py4e, chapter 14, is the free path. It writes `super().__init__()` where these notes write `MenuItem.__init__(self, ...)`, the same idea in a different spelling, and it skips `__str__`, encapsulation and polymorphism, for which the slides and these notes are the reference. For inheritance in depth, the official Python tutorial, chapter 9, "Classes".

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the outputs in sections 2 to 6, and the pictures beside them, and write each one exactly as Python prints it; then look.
- With the picture covered, draw page 19's grid from memory: three spellings down the side, three kinds of code across the top, and in each cell whether it works. Check it against section 3.
- Redo page 25's trace from memory, with the picture covered: write two classes of your own, a base whose `__str__` calls a method and a subclass that overrides it, predict what `print` shows for each, then run them.
- Write a small class from a description, on paper: `__init__` with a default, one method and a `__str__`. Page 16's `Customer` is a good first one. Then type it and run it.
- Explain in one sentence each: `__private` against `_protected`, overriding, and why the register has no `set_cash`. Check against sections 3 and 4.
- With the picture covered, draw section 8's mutable-default picture from memory, two objects sharing one list, and say how to fix it; then draw it again for a class of your own whose default is an empty dictionary.
- In a few days, answer the check-yourself questions again without these notes, then take self-check 7 by hand, with Python closed.

Answers to check yourself

- **1.1** It changes the structure and keeps what the program computes. A test shows the second held: run the old and the new version on the same inputs and assert that the results are equal, as `check_refactor.py` does for the money and the stock.
- **1.2** `Member`, `Session` and `FrontDesk`, and `FrontDesk` owns the bookings: the ask says it alone decides whether a booking goes through.
- **2.1** `25 60`. Each object has its own `watts`: `a` started at the default 40 and was changed to 25, and `b` was built with 60.
- **2.2** `switch_on` has no parameter for the object, and Python always passes one, so the call stops with `TypeError: Lamp.switch_on() takes 0 positional arguments but 1 was given`. The fix is `def switch_on(self):`.

- **3.1** Prices really change, so `MenuItem` offers a door with a rule on it. The cash must change only through a sale, which `sell` checks, so the register offers no door for it at all.
- **3.2** `12.0`. The setter turns `"-5"` into -5.0 and refuses it, so 40 stays; then `"12"` becomes 12.0 and lands.
- **4.1** `fern needs water`, then `spike needs sun`. `Cactus` inherits `__init__` and `__str__`, and the `__str__` it inherits from `Plant` calls `self.needs()`, which finds `Cactus`'s own `needs` first.
- **4.2** It stops with `AttributeError: 'Cactus' object has no attribute 'name'`. The new `__init__` overrides `Plant`'s and never calls it, so `name` is never stored, and the inherited `__str__` needs it. The fix is page 24's first rule: call the base constructor by name, first, then set `spines`.
- **5.1** The 3 scones: accepted, 1 left. The 2 scones: refused, since 1 cannot cover 2, and nothing changes, so 1 is still left. The 1 scone: accepted, 0 left.
- **5.2** No. It compares only the revenue and the stock. The replies' wording changed on purpose, `we don't sell wine` became `REJECT unknown item: wine`, and nothing in the check compares them.
- **6.1** `21.5`. Each `__str__()` gives text, `'2'` and `'1.5'`, and `+` on two texts joins them; it adds only numbers.
- **6.2** `s = Sensor()` works: the `raise` pattern does not stop the object being built. `s.read()` stops with `NotImplementedError: every Sensor subclass must define read()`.
- **7.1** Only the `Ferry`'s own methods, here `board`, can change `__seats_left`. A caller learns the outcome from the pair `board` hands back, and reads the count through `get_seats_left()`.
- **7.2** It replies: a refused order comes back as `(False, "REJECT ...")`, a normal answer, and the caller reads the pair, shows the message and carries on, where a raised error would stop it unless something caught it.
- **8.1** `1 1`. `a` and `c` share the one default list, so `c` sees the `"novel"` that `a` added; `b` was handed a list of its own, `["atlas"]`, and is untouched.
- **8.2** `p._name` gives the name back: one underscore is protected by convention, a request that outsiders stay away, and Python does not enforce it. With `self.__name`, `p.__name` stops with `AttributeError: 'Person' object has no attribute '__name'`, since inside the class the name was stored as `_Person__name`.